UNIVERSIDAD DE MANIZALES
Facultad de Ingeniería — Ingeniería de Sistemas y Telecomunicaciones

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

(Estándar IEEE 830-1998)

Simulador ECG & Análisis Frecuencial (FFT) — "ecg-signal-simulator"

Asignatura: Teoría de Señales
Repositorio: github.com/JeroSoto/teoria_se-ales_ecg

Autores:
Jerónimo Soto Villada
Milady Cristina Guapacha Montoya

Manizales, Colombia — 2026

1. Introducción

1.1 Propósito

Este documento especifica, bajo el estándar IEEE 830-1998, los requisitos funcionales y no funcionales del sistema "ecg-signal-simulator": una plataforma web educativa que simula en tiempo real una señal de electrocardiograma (ECG), permite aplicar perturbaciones y filtros digitales característicos de la teoría de señales, y ofrece un script complementario en Python para el análisis espectral (FFT) de la señal. El documento está dirigido a los docentes evaluadores de la asignatura Teoría de Señales, a los desarrolladores del proyecto y a cualquier colaborador futuro del repositorio.

1.2 Alcance

El producto, identificado como "Simulador ECG & Teoría de Señales", comprende:

- Una aplicación web (Node.js/Express + HTML5 Canvas + JavaScript) que genera y visualiza en tiempo real una forma de onda ECG sintética.
- Un módulo de procesamiento digital de señales (DSP) en el navegador: filtros pasa-altas, pasa-bajas y notch, además de un detector de picos R para estimar la frecuencia cardíaca.
- Integración opcional con un pulsómetro Bluetooth Low Energy (perfil estándar Heart Rate), cuyos BPM reales ajustan el ritmo de la onda sintética.
- Un script independiente en Python (SciPy/NumPy/Matplotlib) que genera una señal ECG sintética, calcula su FFT, aplica filtrado digital y exporta gráficas comparativas al directorio results/.
- Material teórico embebido en la interfaz sobre muestreo, Nyquist-Shannon, contenido frecuencial del ECG y el algoritmo de Pan-Tompkins.

Queda explícitamente fuera del alcance cualquier uso como herramienta de diagnóstico clínico, monitoreo médico real o toma de decisiones sobre la salud de una persona: el sistema es de carácter estrictamente educativo.

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
ECG	Electrocardiograma: registro de la actividad eléctrica del corazón.
BPM	Latidos (beats) por minuto.
FFT	Transformada Rápida de Fourier (Fast Fourier Transform).
DSP	Procesamiento Digital de Señales (Digital Signal Processing).
BLE / GATT	Bluetooth Low Energy / Generic Attribute Profile, protocolo usado para leer el sensor de frecuencia cardíaca.
Notch	Filtro rechaza-banda angosto, aquí centrado en 60 Hz.

Término	Definición
RR	Intervalo de tiempo entre dos picos R (complejos QRS) consecutivos.
SRS	Software Requirements Specification — este documento.

1.4 Referencias

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Repositorio del proyecto: github.com/JeroSoto/teoria_se-ales_ecg (carpeta ecg-signal-simulator).
- README.md y docs/teoria_senales.md del repositorio.
- Pan, J., & Tompkins, W. J. (1985). A real-time QRS detection algorithm. IEEE Transactions on Biomedical Engineering, 32(3), 230-236.

1.5 Visión general del documento

La sección 2 presenta una descripción general del producto: su perspectiva, funciones principales, usuarios, restricciones y supuestos. La sección 3 detalla los requisitos específicos —funcionales, no funcionales y de interfaz externa— derivados directamente del código fuente del repositorio. Los apéndices resumen la estructura de archivos del proyecto.

2. Descripción general

2.1 Perspectiva del producto

El sistema es un producto autocontenido, no sucesor de un sistema previo. Se compone de dos subsistemas independientes que comparten el mismo marco teórico:

- Subsistema web en tiempo real (src/server.js + src/public/): servidor Express que sirve una SPA estática; toda la generación, el filtrado y la detección de picos ocurren en el navegador (cliente).
- Subsistema de análisis offline (scripts/analisis_fft.py): script Python independiente, ejecutado por línea de comandos, que no se comunica con el servidor Node.js.

2.2 Funciones del producto

A alto nivel, el sistema permite:

- Simular una señal ECG sintética realista (ondas P, Q, R, S, T) a una frecuencia de muestreo fija de 250 Hz.
- Contaminar la señal con ruido gaussiano, interferencia de red eléctrica (60 Hz) y deriva de línea base, de forma configurable.
- Filtrar la señal en tiempo real y comparar visualmente el efecto de cada filtro.
- Detectar automáticamente la frecuencia cardíaca a partir de los picos R de la señal procesada.
- Sincronizar la simulación con un pulsómetro Bluetooth real, cuando está disponible.

- Generar, de forma offline, gráficas de la señal en el dominio del tiempo y la frecuencia, antes y después del filtrado.

2.3 Características de los usuarios

Tipo de usuario	Descripción
Estudiante / autor	Usa el simulador para ilustrar y verificar conceptos de teoría de señales (muestreo, Nyquist, FFT, filtrado).
Docente evaluador	Revisa el funcionamiento del simulador y la correspondencia entre el código y la teoría presentada.
Desarrollador colaborador	Da mantenimiento o extiende el código fuente del repositorio.

2.4 Restricciones

- El proyecto es de carácter estrictamente educativo; no debe usarse con fines de diagnóstico clínico (restricción declarada explícitamente en la interfaz y el README).
- La conexión Bluetooth solo es posible en navegadores compatibles con Web Bluetooth (Chrome o Edge) y requiere un contexto seguro (localhost o HTTPS).
- No todos los relojes/pulsómetros exponen sus datos a aplicaciones web, aun cuando midan frecuencia cardíaca.
- El pulsómetro Bluetooth entrega únicamente BPM (no una señal ECG real); la forma de onda mostrada sigue siendo sintética.
- El servidor Node.js requiere versión 18.0.0 o superior; el script de análisis requiere Python 3 con numpy, scipy y matplotlib.

2.5 Supuestos y dependencias

- Se asume que el usuario cuenta con un navegador moderno y, opcionalmente, un sensor BLE compatible con el servicio estándar "heart_rate" (UUID 0x180D).
- El subsistema web depende únicamente de la librería Express (dependencia declarada en package.json).
- El script de análisis depende de NumPy, SciPy y Matplotlib, pero es independiente del servidor web.

3. Requisitos específicos

3.1 Requisitos funcionales

ID	Descripción	Prioridad
RF-01	El sistema debe generar una señal ECG sintética (ondas P, Q, R, S, T mediante funciones gaussianas) muestreada a fs = 250 Hz.	Alta
RF-02	El sistema debe permitir ajustar la frecuencia cardíaca simulada entre 40 y 180 BPM mediante un control deslizante.	Alta

ID	Descripción	Prioridad
RF-03	El sistema debe permitir ajustar la amplitud de la señal entre 0.2x y 2.0x.	Media
RF-04	El sistema debe permitir ajustar el nivel de ruido gaussiano añadido a la señal, de 0 % a 100 %.	Media
RF-05	El sistema debe permitir activar/desactivar una interferencia sinusoidal de red eléctrica a 60 Hz.	Alta
RF-06	El sistema debe permitir activar/desactivar una deriva de línea base (< 0.5 Hz).	Alta
RF-07	El sistema debe permitir conectar un pulsómetro Bluetooth LE que implemente el servicio estándar Heart Rate (0x180D) y leer la característica Heart Rate Measurement (0x2A37).	Alta
RF-08	Cuando un pulsómetro esté conectado, el sistema debe ajustar automáticamente el BPM de la onda sintética al valor recibido del sensor, en lugar del control deslizante.	Alta
RF-09	El sistema debe permitir desconectar el pulsómetro en cualquier momento y reanudar el control manual de BPM.	Media
RF-10	El sistema debe aplicar, de forma independiente y combinable, un filtro pasa-altas (fc ≈ 0.5 Hz), un filtro pasa-bajas (fc ≈ 40 Hz) y un filtro notch (60 Hz) sobre la señal en tiempo real.	Alta
RF-11	El sistema debe detectar automáticamente los picos R de la señal procesada y calcular la frecuencia cardíaca estimada (promedio móvil de los últimos 5 intervalos RR).	Alta
RF-12	El sistema debe renderizar la señal en tiempo real en un lienzo (Canvas) tipo osciloscopio, con cuadrícula de referencia.	Alta
RF-13	El sistema debe mostrar en pantalla, de forma continua: frecuencia cardíaca (lpm), frecuencia fundamental (Hz), intervalo RR (s) y los filtros actualmente activos.	Media
RF-14	El sistema debe permitir pausar y reanudar la simulación.	Baja
RF-15	El sistema debe exponer un endpoint HTTP GET /api/config con los parámetros por defecto del proyecto (nombre, fs, BPM, amplitud, nivel de ruido).	Baja
RF-16	El sistema debe incluir un script independiente en Python que genere una señal ECG sintética, calcule su FFT y aplique filtros digitales (SciPy) equivalentes a los del cliente web.	Alta
RF-17	El script de análisis debe exportar al directorio results/ gráficas comparativas de la señal en el tiempo y en frecuencia, antes y después del filtrado.	Media
RF-18	La interfaz debe presentar contenido teórico embebido (Nyquist-Shannon, contenido frecuencial del ECG, fórmulas de BPM/frecuencia y Pan-Tompkins).	Baja

3.2 Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Rendimiento	La generación, el filtrado y el renderizado de cada muestra deben completarse dentro del período de muestreo (1/250 s ≈ 4 ms) para sostener la visualización en tiempo real.
RNF-02	Usabilidad	La interfaz debe estar completamente en español, con controles accesibles (atributos role="status" y aria-live en los mensajes de estado del sensor).
RNF-03	Compatibilidad	La función de pulsómetro Bluetooth solo debe habilitarse en navegadores con soporte de Web Bluetooth (Chrome, Edge) y en contexto seguro (localhost o HTTPS); en caso contrario, el botón de conexión debe deshabilitarse con un mensaje explicativo.
RNF-04	Portabilidad	El subsistema web debe ejecutarse con Node.js ≥ 18; el subsistema de análisis debe ejecutarse con Python 3 y las librerías numpy, scipy y matplotlib.
RNF-05	Mantenibilidad	El código cliente debe mantenerse modularizado en componentes independientes (generador, procesador DSP, renderizador, monitor Bluetooth y orquestador de interfaz).
RNF-06	Fiabilidad	Un error o cancelación en la conexión Bluetooth no debe interrumpir la simulación: el sistema debe degradarse de forma controlada al modo manual.
RNF-07	Seguridad / privacidad	La conexión al sensor requiere autorización explícita del usuario a través del selector nativo del navegador; el sistema no debe almacenar de forma persistente datos del sensor.
RNF-08	Licenciamiento	El proyecto debe distribuirse bajo licencia MIT, permitiendo su reutilización y modificación.
RNF-09	Alcance ético	El sistema debe declarar explícitamente, en la interfaz y en la documentación, que su uso es exclusivamente educativo y que no constituye una herramienta de diagnóstico clínico.

3.3 Interfaces externas

3.3.1 Interfaces de usuario

- Aplicación web de una sola página (src/public/index.html), con panel de monitor (Canvas + métricas), panel de controles de simulación/ruido, panel de conexión Bluetooth y sección teórica.

3.3.2 Interfaces de hardware

- Sensor de frecuencia cardíaca Bluetooth Low Energy (p. ej. banda pectoral) que implemente el perfil GATT estándar "heart_rate" (servicio 0x180D, característica 0x2A37).

3.3.3 Interfaces de software

- Backend: Node.js + Express (servidor HTTP de archivos estáticos y del endpoint /api/config).
- Frontend: HTML5, CSS3 y JavaScript Vanilla (sin frameworks), con HTML5 Canvas para el renderizado de la señal.

- Análisis offline: Python 3, NumPy, SciPy (scipy.signal) y Matplotlib.

3.3.4 Interfaces de comunicación

- HTTP sobre el puerto 3000 (configurable por la variable de entorno PORT) entre el navegador y el servidor Express.
- Web Bluetooth API (GATT sobre BLE) entre el navegador y el sensor de frecuencia cardíaca.

4. Apéndices

4.1 Estructura del repositorio

Ruta	Contenido
src/server.js	Servidor HTTP Express; expone /api/config y sirve el frontend estático.
src/public/js/ecgGenerator.js	Generación de la forma de onda ECG sintética (ondas P-QRS-T) y del ruido.
src/public/js/signalProcessing.js	Filtros pasa-altas, pasa-bajas y notch; detección de picos R y cálculo de BPM.
src/public/js/renderer.js	Renderizado de la señal en Canvas (osciloscopio).
src/public/js/heartRateMonitor.js	Conexión y lectura del pulsómetro Bluetooth (Web Bluetooth API).
src/public/js/app.js	Orquestador: conecta la UI con el generador, el procesador y el renderizador.
src/public/index.html / css/styles.css	Interfaz de usuario y estilos.
scripts/analisis_fft.py	Script offline: genera señal, calcula FFT, filtra con SciPy y exporta gráficas.
results/	Gráficas generadas por el script de análisis.
docs/teoria_senales.md	Apuntes teóricos de la asignatura.
data/muestra_ecg_sintetico.json	Muestra de datos ECG sintéticos.
package.json	Dependencias y scripts npm (start, dev, analyze).